Capstone Project: Quantitative Strategy Research

Data Structures and Algorithms, Imperial College Business School

Overview

The capstone is an individual, open-ended project completed at the end of the module. You will pose your own hypothesis about a market inefficiency or pattern, acquire and clean real data, design and implement a trading strategy, and backtest it. This is the same core workflow used in quantitative research roles.

Use of GenAI tools (Claude, ChatGPT, Copilot, etc.) and any Python packages is explicitly allowed and expected. What's assessed is not whether you typed every line yourself, but whether **you understand and can defend every part of your pipeline**, and whether your idea is genuinely yours.

What you're building

Five things, submitted together:

1. **A hypothesis pitch** (~half a page)
2. **A Jupyter notebook**: data pipeline, strategy, backtest
3. **A written report**: methodology, results, and critical reflection
4. **A completed pitfall checklist** (part of the report; see below)
5. **A reflection** (part of the report; see below)

1. Hypothesis pitch

Before writing code, state:

- **What** pattern or inefficiency you believe exists (e.g. "stocks with X characteristic tend to Y over Z horizon")

- **Why** you think it might be true: an economic or behavioural mechanism, not just "I noticed a pattern." If you can't articulate *why* it might work, you don't have a hypothesis yet; you have a curve fit waiting to happen
- **What would prove you wrong**: what result would make you abandon the idea

Avoid generic, reflexive ideas. A simple moving-average crossover on a major index, or "buy and hold beats random," will not pass this section unless you can argue a genuinely novel angle on it. You're welcome to brainstorm with GenAI to get to your idea (that's a legitimate way to work), but the idea you land on has to move past whatever the first, most obvious suggestion was. If GenAI's first suggestion sounds generic, that's a sign to push the conversation further, not a sign you've found your idea. Strong pitches usually end up with a specific, personal angle on a market, instrument, or dataset.

2. Data pipeline

- Real data, from a source of your choice: a live API (e.g. `yfinance`), a downloaded dataset, or a non-public/paid source if you have access to one
- Not limited to price and volume: you're encouraged to bring in other kinds of data where relevant to your hypothesis, such as fundamentals, news/sentiment, macroeconomic series, alternative data, or anything else that supports your idea
- If your data source is non-public (a paid API, a private dataset, etc.), you must still submit a downloaded snapshot of the data you used, and your full data pipeline code, regardless of source. Someone should be able to see exactly how your raw data became your model inputs without needing your access.
- If a data source sits behind authentication and genuinely can't be automated (e.g. a manual export from a terminal or portal), a manual download is completely fine: just include the downloaded file and document how you obtained it. That said, if you *can* automate access to a private/authenticated source (e.g. scripting an authenticated API call), that's a nice-to-have that will be credited; it's a stronger demonstration of pipeline skill than a one-off manual export. For public data, automation is usually straightforward and is expected by default.

> **Remove any private API keys or credentials from your submission before handing it in.**

- Real-world messiness must be handled, not silently dropped: missing days, corporate actions (splits/dividends), and, if you use a static universe of assets, awareness of

survivorship bias (see checklist below)
- Any universe: equities, crypto, FX, or otherwise, as long as it supports your hypothesis

3. Strategy and backtest

- Fully open-ended: momentum, mean-reversion, pairs/statistical arbitrage, graph-based portfolio construction, a simple ML-based signal, or something else entirely
- Use any backtesting approach or library you like
- Standard performance metrics: at minimum, cumulative return, Sharpe ratio, and maximum drawdown, evaluated **out-of-sample** (a held-out test period or walk-forward validation, not the same data you used to design/tune the strategy)

4. Methodology defense (in the report)

For each major choice in your pipeline, briefly justify it: why this universe, why this train/test split, why these transaction cost assumptions, why this particular signal construction. This is the section that reveals whether you understand what your own code is doing. If you used AI to write a component, you should still be able to explain *why* it's built that way, and what would break if a key assumption were wrong.

5. Pitfall checklist (required, in the report)

For each of the following, state how you checked for it in your own pipeline, and what you found. "Not applicable, because..." is an acceptable answer where genuinely true, but it must be argued, not asserted.

- [ ] **Look-ahead bias**: does your strategy ever use information that would not have been available at the decision time it's being applied?
- [ ] **Survivorship bias**: if your universe is a fixed list of assets, does it exclude companies that were delisted, went bankrupt, or were acquired during your sample period? How would including them change your results?
- [ ] **Overfitting / data-snooping**: did you tune parameters and evaluate performance on the same data? How many variations did you try before settling on this one?
- [ ] **Transaction costs and slippage**: does the strategy still look attractive after realistic trading costs? What assumption did you use, and why?
- [ ] **Out-of-sample validation**: what period(s) were held out from strategy design, and how did performance there compare to the in-sample period?

6. Efficiency touchpoint (required, brief)

Identify the single most computationally expensive step in your pipeline. State its time complexity (for example, pairwise correlation across `n` assets costs `O(n^2)`) and justify why that's acceptable at your scale, or describe how you avoided/reduced it. This applies regardless of whether you or an AI tool wrote the underlying code: you still need to understand and be able to explain what it's doing computationally.

7. Risk touchpoint (required, brief)

Briefly characterise the risk of your strategy, beyond just the Sharpe ratio and max drawdown already required in your backtest results: position sizing/leverage, concentration (how diversified is it really?), and what a plausible bad scenario would do to it (a volatility spike, a correlation breakdown, a liquidity crunch, whichever is most relevant to your strategy). A couple of sentences is enough; the point is to show you've thought about how your strategy could lose money, not just how it made money in the backtest.

8. Reflection (required, in the report)

Close your report with a short, honest reflection:

- What did you learn that you didn't know before starting?
- What was wrong in your original perception or assumptions, about your hypothesis, the data, or the market itself, that only became clear once you actually built and tested it?
- What are the natural next steps: extending the idea, testing it on a different universe/period, scaling it up to more capital or more assets, or moving it toward something closer to production? What would need to change to get there?

This section is not scored on whether your strategy "worked." A negative result is completely fine: "I applied a genuinely sophisticated approach here, and it still didn't hold up out-of-sample" is a legitimate, valuable finding, and a well-reasoned account of why a hypothesis *failed* is just as valuable as a successful one. What's not fine is a negative result from a trivial or careless methodology, where the failure tells you nothing because the approach was never a fair test of the idea in the first place.

Submission format

To keep review consistent and fair across all submissions, please structure your work exactly as follows:

- **Three items, clearly separated**: `report` (PDF or Markdown), `notebook.ipynb`, and a `data/` folder containing your downloaded data snapshot. Don't bury the report inside the notebook as scattered markdown cells; it should be a single, standalone, readable document.
- **Start the report with your name, submission date, and a one-line project title.** No separate title page needed, just these at the top of page one.
- **Use the exact section headings and numbering from this brief**, in the same order, as headings in your report: `1. Hypothesis Pitch`, `2. Data Pipeline`, `3. Strategy and Backtest`, `4. Methodology Defense`, `5. Pitfall Checklist`, `6. Efficiency Touchpoint`, `7. Risk Touchpoint`, `8. Reflection`. This makes your work easy to navigate and makes sure nothing you wrote gets accidentally overlooked.
- **Within the Pitfall Checklist section, use the five pitfall names as their own subheadings** (Look-ahead bias, Survivorship bias, Overfitting/data-snooping, Transaction costs and slippage, Out-of-sample validation), each followed directly by your answer, rather than addressing them in one continuous paragraph.
- **State your key results as an explicit labeled list or small table** near the start of your Results discussion (cumulative return, Sharpe ratio, max drawdown, in-sample vs. out-of-sample); don't make a reader hunt for these numbers inside a chart or a paragraph.
- **In the notebook, add a short markdown heading before each major stage** (data acquisition, cleaning, strategy, backtest) so the structure is clear at a glance, matching the order it's described in your report.
- **If you cite any external sources** (a paper, article, or methodology that informed your hypothesis or data source), list them in a short `References` section at the end of your report. Not every project will have these; only include it if genuinely applicable.

Deliverables checklist

To pass, your submission must include:

- [ ] Hypothesis pitch (half a page) that is genuinely yours by the end. You may have brainstormed it with GenAI, but you must be able to explain and defend it in your own terms; this is assessed via the quality and specificity of your "why" in the pitch and your methodology defense, not by policing tool use
- [ ] Jupyter notebook: data acquisition → cleaning → strategy → backtest, runs end-to-end without errors

- [ ] Downloaded snapshot of your data, and your full data pipeline, even if your source is non-public (with any private keys/credentials removed)
- [ ] Written report covering: methodology defense, results, pitfall checklist (all five items addressed), efficiency touchpoint, risk touchpoint, and reflection

A note on GenAI use

Use whatever tools help you work efficiently; that mirrors how this work actually gets done in industry, including brainstorming your initial idea with GenAI. What can't be delegated is the thinking: the hypothesis has to end up genuinely yours, and you have to be able to explain and defend every methodological choice in your own words, whether or not AI helped write the code that implements it. Submissions that show clear signs of the student not understanding their own pipeline (e.g. inability to explain a design choice, or a pitfall-checklist answer that doesn't match what the code actually does) will not pass regardless of how polished the notebook looks.